Python Exercise Solutions

Chapter: Manipulate data with standard libraries and co-locate code with classes and functions

1. **Customer Order Analysis:** Write python code that processes a list of customer orders to calculate the total revenue and find top 3 the most frequent customer.

We have a list of orders, where each order is a dictionary with keys: customer_id, product, quantity, and price.

`revenue = quantity * price`

frequency of customer is defined as the number of orders

```
orders = [
    {"customer_id": "C001", "product": "laptop", "quantity": 2, "price": 1200.00},
    {"customer_id": "C002", "product": "mouse", "quantity": 1, "price": 25.99},
    {"customer_id": "C001", "product": "keyboard", "quantity": 1, "price": 89.50},
    {"customer_id": "C003", "product": "monitor", "quantity": 1, "price": 299.99},
    {"customer_id": "C002", "product": "laptop", "quantity": 1, "price": 1200.00},
    {"customer_id": "C004", "product": "headphones", "quantity": 3, "price": 79.99},
    {"customer_id": "C001", "product": "webcam", "quantity": 1, "price": 45.00},
    {"customer_id": "C003", "product": "mouse", "quantity": 2, "price": 25.99},
    {"customer_id": "C002", "product": "speaker", "quantity": 1, "price": 150.00},
    {"customer_id": "C005", "product": "tablet", "quantity": 1, "price": 399.99}
]
```

```
# Total revenue
total_revenue = 0
for order in orders: # loop through each order
    total_revenue += (order['quantity'] * order['price']) # add the revenue of each order to total_revenue

print(f'Total revenue is {round(total_revenue, 2)}')
```

```python
# Top 3 frequent customer
customer_order_count = {}

for order in orders: # loop through each order
    if order['customer_id'] in customer_order_count: # add 1 to existing customer key, else assign to 1. Note Look into defaultdict to make this simpler
        customer_order_count[order['customer_id']] += 1
    else:
        customer_order_count[order['customer_id']] = 1

sorted_by_revenue = sorted(customer_order_count.items(), key=lambda x: x[1], reverse=True) # this sorts our dictionary based on x[1] which corresponds to
# values from key,values generated by the items function, and we sort in decreasing order with reverse
print(sorted_by_revenue)
```

2. Data Quality Checker: Write a Python function that takes a list of email addresses and returns a dictionary with two keys: valid_emails (list) and invalid_emails (list).

Use basic validation rules

1. must contain `@`
2. `.` must be after `@`
3. must contain text before the `@`

```python
email_list = [
    "john.doe@company.com",
    "jane.smith@email.co.uk",
    "invalid-email", # incorrect email
    "bob@gmail.com",
    "alice.brown@company.com",
    "john.doe@company.com",  # duplicate
    "missing@domain", # incorrect email
    "test@example.org",
    "@nodomain.com", # incorrect email
    "jane.smith@email.co.uk",  # duplicate
    "valid.user@site.net",
    "no-at-symbol.com", # incorrect email
    "another@test.io"
]
```

```
'@something'.split('@')
```

```
valid_emails = []
invalid_emails = []
for email in email_list:
    split_email = email.split('@')
    identifier = split_email[0]
    domain = None
    org = None
    if len(split_email) > 1:
        domain_org = split_email[1]
        domain = domain_org.split('.')[0]
        org = domain_org.split('.')[1] if len(domain_org.split('.')) > 1 else None

    if identifier != '' and domain and org:
        valid_emails.append(email)
    else:
        invalid_emails.append(email)
```

```
valid_emails
```

```
invalid_emails
```

Explanation:

1. Splits email at '@' - separates local part from domain part
2. Extracts identifier - everything before '@' (like "user" in "user@example.com")
3. Splits domain part at '.' - separates domain from organization (like "example" and "com")
4. Validates all parts exist - checks that identifier, domain, and org are all non-empty
5. Sales Performance Tracker: Create a class called SalesRep that stores a representative's name and a list of their sales amounts.

Include methods to add sales amounts, calculate average sales, and determine if they hit a target (parameter).

```
# Sample data for creating SalesRep objects
sales_data = {
    "Alice Johnson": [15, 18, 22, 16, 19, 21],
    "Bob Smith": [12, 14, 11, 13, 15, 16],
    "Carol Davis": [25, 28, 30, 27, 32, 29, 100]
}
```

```
class SalesRep:

    def __init__(self, name):
        self.name = name
        self.sales_amount = []

    def add_monthly_sales(self, sales_amount):
        self.sales_amount.append(sales_amount)

    def get_average_sales(self):
        return avg(self.sales_amount)

    def hit_target(self, target):
        return sum(self.sales_amount) > target
```

```
for rep_name, sales_list in sales_data.items():
    sales_rep = SalesRep(rep_name)
    target = 200
    for sale in sales_list:
        sales_rep.add_monthly_sales(sale)
    print(f' Did {sales_rep.name} hit sale target of {target}? :
    ↪ {sales_rep.hit_target(target)}')
```

Chapter: Python has libraries to read and write data to (almost) any system

1. Fetch Data from API and Save Locally. Pull Pokemon data from the PokeAPI and save it to a local file. The local JSON file should contain Pokemon data for Bulbasaur (ID: 1)

```
import requests
import json

data_api = "https://pokeapi.co/api/v2/pokemon/1/"
```

```
local_file = "pokemon_data.json"

# 1. Make a GET request to data_api
response = requests.get(data_api)
response.raise_for_status()  # Raises an HTTPError for bad responses

# 2. Extract the JSON response
pokemon_data = response.json() # response has a convenient json method for this

# 3. Write the JSON data to local_file
with open(local_file, 'w') as file:
        json.dump(pokemon_data, file, indent=2) # If we do not use indent, the json will be dumped into a single line in the local_file
```

```
! cat {local_file} | head -10 # this prints the first 10 lines of the local_file using unix commands
```

2. Read and Display Local File Contents. Read the previously saved JSON file and print its name and id.

```
import json

# 1. Use Python standard libraries to open local_file and dump the contents into a json
with open(local_file, 'r') as file:
    pokemon_data = json.load(file)

# 2. Read the name and id from the json
print(pokemon_data['name'], pokemon_data['id'])
```

3. Parse Data and Insert into SQLite Database. Extract specific Pokemon attributes and store them in a SQLite database.

Table Schema:

```
CREATE TABLE pokemon (
    id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    base_experience INTEGER
);
```

```python
import sqlite3
import json

local_file = "pokemon_data.json"
database_file = "pokemon.db"


# 1. Create a SQLite3 connection and table with columns: id, name, base_experience
conn = sqlite3.connect(database_file)
cursor = conn.cursor()

# Create table with specified schema
# We drop and recreate table each time, note this is probably not what you'd want to do in real project
cursor.execute("DROP TABLE IF EXISTS pokemon")
cursor.execute('''
    CREATE TABLE IF NOT EXISTS pokemon (
        id INTEGER PRIMARY KEY,
        name TEXT NOT NULL,
        base_experience INTEGER
    )
''')

# Commit the command execution
conn.commit()

# 2. Open and read the local_file using Python standard libraries
with open(local_file, 'r') as file:
    pokemon_data = json.load(file)

# 3. Parse the JSON data to extract id, name, and base_experience
pokemon_id = pokemon_data['id']
pokemon_name = pokemon_data['name']
base_experience = pokemon_data['base_experience']

# 4. Insert the extracted data into the SQLite table
cursor = conn.cursor()

# Use INSERT
cursor.execute('''
    INSERT INTO pokemon (id, name, base_experience)
    VALUES (?, ?, ?)
```

```
''', (pokemon_id, pokemon_name, base_experience))
# always commit to ensure that the changes you made are pushed into the db (from in memory)
conn.commit()

# 5. Verify insertion by querying and printing the data
cursor = conn.cursor()
cursor.execute("SELECT * FROM pokemon")
rows = cursor.fetchall()
print(rows)
```

Chapter: Python has libraries to tell the data processing engine (Spark, Trino, Duckdb, Polars, etc) what to do

Run the below cell before getting started with the exercise

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, coalesce, lit, when, split, sum as spark_sum, avg, regexp_replace
from pyspark.sql.types import StructType, StructField, IntegerType, StringType, FloatType, DateType

schema = StructType([
    StructField("Customer_ID", IntegerType(), True),
    StructField("Customer_Name", StringType(), True),
    StructField("Age", IntegerType(), True),
    StructField("Gender", StringType(), True),
    StructField("Purchase_Amount", FloatType(), True),
    StructField("Purchase_Date", DateType(), True)
])

# Read data from CSV file into DataFrame
data = spark.read \
    .option("header", "true") \
    .option("inferSchema", "false") \
    .schema(schema) \
    .csv("./sample_data.csv")

# Question: How do you remove duplicate rows based on customer ID in PySpark?
data_unique = data.dropDuplicates()

```

```
# Question: How do you handle missing values by replacing them with 0 in PySpark?
data_cleaned_missing = data_unique.select(
    col("Customer_ID"),
    col("Customer_Name"),
    coalesce(col("Age"), lit(0)).alias("Age"),
    col("Gender"),
    coalesce(col("Purchase_Amount"), lit(0.0)).alias("Purchase_Amount"),
    col("Purchase_Date")
)

# Question: How do you remove outliers (e.g., age > 100 or purchase amount > 1000) in PySpark?
data_cleaned_outliers = data_cleaned_missing.filter(
    (col("Age") <= 100) & (col("Purchase_Amount") <= 1000)
)

# Question: How do you convert the Gender column to a binary format (0 for Female, 1 for Male) in PySpark?
data_cleaned_gender = data_cleaned_outliers.withColumn(
    "Gender_Binary",
    when(col("Gender") == "Female", 0).otherwise(1)
)

# Question: How do you split the Customer_Name column into separate First_Name and Last_Name columns in PySpark?
data_cleaned = data_cleaned_gender.select(
    col("Customer_ID"),
    split(col("Customer_Name"), " ").getItem(0).alias("First_Name"),
    split(col("Customer_Name"), " ").getItem(1).alias("Last_Name"),
    col("Age"),
    col("Gender_Binary"),
    col("Purchase_Amount"),
    col("Purchase_Date")
)
```

1. What is the total purchase amount for each gender in the dataset? Use the `data_cleaned_gender` dataframe and Group the data by gender and calculate the sum of all purchase amounts.

```
print("Total purchase by gender:")
data_cleaned.groupBy("Gender_Binary") \
```

```
    .agg(spark_sum("Purchase_Amount").alias("Total_Purchase_Amount")) \
    .show()
```

2. What is the average purchase amount for different age groups? Use the `data_cleaned` dataframe and create age group categories (18-30, 31-40, 41-50, 51-60, 61-70) and calculate the mean purchase amount for each group.

```
print("Average purchase by age group:")
data_cleaned.withColumn(
    "Age_Group",
    when((col("Age") >= 18) & (col("Age") <= 30), "18-30")
    .when((col("Age") >= 31) & (col("Age") <= 40), "31-40")
    .when((col("Age") >= 41) & (col("Age") <= 50), "41-50")
    .when((col("Age") >= 51) & (col("Age") <= 60), "51-60")
    .otherwise("61-70")
).groupBy("Age_Group") \
    .agg(avg("Purchase_Amount").alias("Average_Purchase_Amount")) \
    .show()
```